Delim

This project may appeal to programmers who dislike the indentation-based structure in CoffeeScript but otherwise like the language for its simplicity, terseness and expressiveness.

Although CoffeeScript was influenced by Python and Ruby, it uses neither Python's colon nor Ruby's **end** keyword. ***Delim*** reintroduces both to clearly delimit code blocks.

1) A code block is opened with a trailing colon and is terminated with a **end** keyword.
2) Exceptionally, a **switch** construct must be terminated with **end switch**.
3) A function block is still opened with a trailing arrow. A colon is allowed but it is redondant.
4) A one-liner still uses the CoffeeScript's **then** keyword, not a colon.

This offers some advantages:
- Whitespace is no longer significant.
- Some people feel more comfortable with the structural integrity of delimited blocks.
- Safe copy-pasting without worrying about indentation levels.
- Safe auto-formatting.

Syntax restrictions

Delim is a text preprocessor without a stack or a lexer. It works mostly on a line-by-line basis.
As a result, it has a few syntax restrictions, which are listed below. Fortunately, they align with recommended coding practices and help improve readability.

- No comment is allowed after a trailing colon or a trailing arrow.
- Only use code blocks throughout a multi-line **if**, **switch**, or **try** constructs.

Usage

Delim outputs to *stdout*. The option *--format* reindents lines consistently, the code remains unchanged.

delim [*--format*] <script>.coffee

Typical usage

The CoffeeScript compiler must be installed to get JavaScript.

```
# Convert and compile to JavaScript in one step
delim script.coffee | coffee -s -c > script.js

# Get a converted copy of a source
delim script.coffee > script.converted.coffee

# Get a formatted copy of a source
delim --format script.coffee > script.formatted.coffee
```

General syntax

```
functionName = (parms) ->
 code
end

functionName = (parms) -> code

do ->
 code
end

do -> code

[variable =] if <condition>:
 code
else if <condition>:
 code
else:
 code
end

[variable =] if <condition> then code [else code]

code if <condition>

[variable =] while <condition> [guard clauses]:
 code
end

[variable =] while <condition> [guard clauses] then code

[variable =] for <iterable> [guard clauses]:
 code
end

[variable =] for <iterable> [guard clauses] then code

[variable =] switch [variable]:
  when <condition>:
    code
  [when <condition>:
    code]
  [else:
    code]
end switch

try:
  code
[catch [var]:
  code]
[finally:
  code]
end

class name:
  code
end
```